MANIFOLD

From Writing Prompts to Coding Agents

A Specification-Driven Architecture for Reliable Multi-Agent LLM Systems

Fabio-Eric Rempel
Independent Researcher

February 2026

White Paper v1.0

1. The Problem

Every day, thousands of companies deploy LLM-powered agents to automate tasks: extracting data from documents, generating content, processing customer requests, orchestrating multi-step workflows. And every day, these systems fail in ways that are both predictable and expensive.

The failures follow a pattern. An agent is given a task through a natural language prompt. It produces output. Something is wrong with the output, so the system retries. The retry fails for the same reason, because nothing about the situation has changed. The system retries again. And again. Each attempt costs money, burns time, and produces the same broken result. This is the **infinite retry loop**, and it is the default failure mode of every LLM orchestration system that uses retry logic without progress detection.

But there is a second, more insidious failure that most teams never even discover. When an agent produces output, someone has to decide whether that output is correct. In most current systems, the agent itself makes that judgment. It checks its own work against its own understanding of what was asked. And here is the problem: **agents are unreliable judges of their own output.**

In controlled experiments across 600 trials, we found that standard prompting approaches report 100% success while actually achieving only 34% correct output on structured tasks. The gap—a **66% false positive rate**—means that two out of every three outputs the system considers "successful" are, in fact, wrong. These silent failures cascade downstream: bad extractions feed bad decisions, incorrect generations mislead users, and the entire promise of autonomous AI agents collapses under the weight of outputs nobody can trust.

This whitepaper describes a different approach. Rather than treating prompts as instructions and hoping agents follow them correctly, we treat prompts as **code**—with defined inputs, expected outputs, and verifiable contracts that exist outside the agent's control. The result is Manifold, a specification-driven architecture that eliminates false positives entirely, prevents infinite retry loops, and reduces costs by up to 64%.

But to understand why Manifold works, you first have to understand a shift in thinking about what prompts are and what they can be.

2. The Paradigm Shift: From Instructions to Code

There is a fundamental misconception at the heart of how most people think about LLM agents. The prevailing model is conversational: you *tell* the agent what to do, in natural language, the

same way you would instruct a colleague. “Please extract the sender’s name, email address, and subject from this message.” “Generate a 64x64 sprite of a medieval knight facing right.” The agent reads these instructions, interprets them, and produces output.

This model feels natural. It is also the source of nearly every reliability problem in LLM agent systems.

2.1 The Instruction Illusion

When you give a human colleague instructions, there is an implicit contract: they understand the context, they share your definition of “correct,” and they will ask clarifying questions if something is ambiguous. LLMs have none of these properties. An LLM does not understand your intent—it predicts the most likely continuation of a text sequence. It does not share your definition of correct—it has been trained on a distribution of outputs, many of which would not meet your standards. And it will not ask for clarification—it will confidently produce output that looks plausible regardless of whether it satisfies your actual requirements.

The instruction model has a deeper problem: it places the agent in a dual role. The agent is both **executor** and **judge**. It performs the task and then evaluates its own performance. This is equivalent to asking a student to grade their own exam. The incentive structure is wrong, and the results are predictable: inflated success rates and undetected failures.

There is a third failure that instruction-based approaches cannot escape: **uncovered blind spots**. No matter how thorough your prompt is, there will always be something you overlooked or something outside your control—an edge case in the input format, an assumption about the data that does not hold, a dependency you forgot to account for. With instructional prompts, these blind spots are silent. The agent does not know what it does not know, and neither do you—until the output is wrong in a way nobody anticipated. With specification-driven approaches, these gaps surface immediately. A missing piece of context that the manifest references will produce an explicit input-output error. A validation step that cannot complete because a required field was never defined will halt the workflow with a clear, diagnosable failure. The system does not silently proceed with incomplete information—it forces you to confront what you missed before it becomes a downstream problem.

> *The instruction model treats prompts as requests. The specification model treats prompts as contracts. This is not a stylistic preference—it is the difference between a system that reports 100% success and one that actually achieves it.*

2.2 Prompts as Code

Software engineers solved this problem decades ago. Functions have type signatures. APIs have schemas. Databases have constraints. In every mature engineering discipline, the contract

between a producer and consumer is defined externally, formally, and verifiably. The producer does not get to decide whether its output is valid—the contract does.

The core insight behind Manifold is that **prompts can work the same way**. Instead of writing a prompt that *asks* an agent to do something, you write a specification that *defines* what valid execution looks like. The specification is not a suggestion or a guideline—it is a constraint boundary that the agent cannot break out of on its own, because the verification happens externally.

Consider the difference:

The Instruction Approach

```
Prompt: "Extract the sender name, email, and subject
        from this email. Return JSON."

Validation: Agent checks its own output.
Result: Agent says "done" → system accepts it.
```

The Specification Approach

```
Specification:
  Input: raw email text (string, non-empty)
  Output: JSON object with required fields:
    - sender_name: non-empty string
    - sender_email: valid email format
    - subject: non-empty string
  Postconditions:
    - All fields present and correctly typed
    - Email matches regex pattern
    - No fields contain placeholder text
  Validation: External spec engine, not the agent.
```

The difference is not merely one of detail. In the instruction approach, the agent interprets what "correct" means. In the specification approach, correctness is defined externally and evaluated mechanically. The agent's opinion about whether it succeeded is irrelevant—either the output satisfies the specification, or it does not.

This is what we mean by **coding the prompt rather than writing it**. You are not composing prose for an AI to interpret—you are defining a contract that constrains the agent's behavior space. The agent operates within a clearly defined ruleset that it cannot break out of by its own judgment, because the rules are enforced externally.

2.3 The Evolution of Prompt Engineering

This shift did not happen overnight. The way practitioners interact with LLMs has evolved through three distinct phases, each addressing the limitations of the previous one:

Phase 1: Monolithic Prompts

The earliest approach: one large prompt that contains every instruction, constraint, and example. “Do A, then B, then C. Make sure X. Don’t do Y.” This works for simple tasks but collapses under complexity. The prompt becomes unmanageable, the agent loses track of instructions buried deep in the text, and there is no way to verify intermediate steps.

Phase 2: Sequential Chains

The first improvement: break the monolith into a chain of smaller prompts, each handling one step. Agent A extracts data, passes it to Agent B for validation, then Agent C formats the output. This is better—each step is simpler—but the connections between steps are still implicit. If Agent A produces malformed output, Agent B discovers the problem only when it tries to process it. There are no contracts between the steps, just assumptions.

Phase 3: Agent Contracts (Manifold)

The approach described in this whitepaper: each step in a workflow has explicit specifications that define what it needs (preconditions), what it must produce (postconditions), what must always be true (invariants), and what must change after a failure (progress conditions). These specifications are enforced externally by an orchestrator, not by the agents themselves. The workflow is defined declaratively in a manifest—a data structure that describes the graph of possible steps and transitions—rather than imperatively in code.

Each phase represents a shift in where control lives. In Phase 1, control is in the prompt. In Phase 2, control is in the chain logic. In Phase 3, control is in the specifications—external, verifiable, and agent-independent.

	Monolithic	Chains	Contracts
Control location	Inside prompt	In chain code	In specifications
Validation	Self-reported	Next step discovers	External enforcement
Retry handling	None	Blind retry	Progress-gated
Loop prevention	None	Retry counter	Fingerprint detection
Trust model	Trust the agent	Trust the chain	Trust the contract

Table 1: The evolution from monolithic prompts to agent contracts.

3. How Specifications Work

A specification in Manifold is a pure evaluation function. It takes the current workflow context and an optional candidate output, and returns a structured verdict: pass or fail, with a human-

readable explanation and a concrete suggested fix. Specifications have no side effects, perform no I/O, and do not modify state. They are deterministic given the same inputs.

This purity is essential. Because specifications are side-effect free, they can be evaluated safely at any point—before a step runs, after it completes, during routing decisions, or as part of post-hoc analysis. They are trustworthy in a way that agent self-assessment is not, precisely because they operate outside the agent's influence.

3.1 The Four Specification Categories

Manifold organizes specifications into four categories, each serving a distinct purpose in the workflow lifecycle:

Preconditions

Preconditions define what must be true *before* an agent runs. They gate execution: if preconditions are not met, the step does not execute. This prevents agents from attempting work they cannot complete—for example, trying to extract data from an input that doesn't exist, or calling an API without required credentials. Preconditions catch configuration errors and missing dependencies before they become runtime failures.

Postconditions

Postconditions define what must be true about the output *after* an agent runs. They are the core mechanism for eliminating false positives. A postcondition might verify that extracted JSON contains all required fields, that field values match expected formats, that no placeholder text remains, or that generated content meets minimum quality thresholds. Postconditions are evaluated externally—the agent's self-assessment is not consulted.

Invariants

Invariants define what must *always* be true across the entire workflow run. They act as global safety constraints: the budget has not been exceeded, no forbidden tools have been called, required context has not been corrupted. Invariants are checked at every step transition, providing continuous assurance that the workflow remains within acceptable bounds.

Progress Conditions

Progress conditions are the key mechanism for preventing infinite retry loops. After a step fails and is retried, progress conditions verify that *something has actually changed*. If the situation is identical to the previous attempt—same input, same context, same failure conditions—the progress condition fails and the retry is blocked. This prevents the most expensive failure mode in LLM systems: doing the same thing repeatedly and expecting different results.

> *Specifications are to agents what type systems are to functions. They do not make agents smarter—they make agent failures detectable, debuggable, and recoverable.*

3.2 The Manifest: Workflows as Data

Manifold workflows are defined in YAML or JSON manifests rather than imperative code. A manifest declares the steps in the workflow, the agents responsible for each step, the specifications attached to each step, the possible transitions between steps, and the conditions under which each transition is taken.

This declarative approach has a practical consequence: you can change the workflow's structure, its validation rules, and its routing logic without modifying any code. The agents, the specifications, and the orchestrator are all separate concerns. You swap domains by swapping manifests and spec libraries—not by rewriting the runtime.

```
# Simplified manifest example
steps:
  extract:
    agent: extraction_agent
    pre_specs:  [has_input_text]
    post_specs: [valid_json, all_fields_present, no_placeholders]
    progress_specs: [situation_changed]

  validate:
    agent: validation_agent
    pre_specs:  [has_extracted_data]
    post_specs: [all_fields_valid]

edges:
  - from: extract  → to: validate   (when: post_ok)
  - from: extract  → to: extract    (when: failed, attempts < 3)
  - from: validate → to: __complete__ (when: post_ok)
```

In this example, the extract step cannot run unless input text exists (precondition). Its output must be valid JSON with all required fields and no placeholder text (postconditions). If it fails and retries, it must show progress (progress condition). The orchestrator enforces all of this externally—the extraction agent has no say in whether its output is accepted.

4. Loop Prevention and Trust

4.1 Why Retry Logic Fails

> *"The definition of insanity is doing the same thing over and over and expecting different results." This sentence was running through my head when I first encountered the retry problem in multi-agent systems. It captures, in a single line, why conventional retry logic is fundamentally broken—and why progress detection is not optional.*

The standard approach to handling LLM failures is straightforward: if the output is bad, try again. Most frameworks implement this as a retry counter—try up to N times, then give up. This approach has two critical flaws.

First, it retries without requiring progress. If the agent failed because the prompt was ambiguous, retrying with the same prompt will produce the same failure. The retry burns compute and money but changes nothing. In adversarial domains—where the LLM actively resists structured constraints, as with certain image generation models—retry logic can inflate costs by 1.5 to 3.5 times without improving success rates.

Second, retry counters do not prevent semantic duplication. An agent might produce three different outputs that all fail for the same underlying reason. A counter counts attempts, not progress. It cannot distinguish between "tried something new and failed" and "tried the exact same thing and failed again."

4.2 Fingerprint-Based Loop Detection

Manifold addresses this with fingerprint-based loop detection. Each attempt generates a semantic fingerprint of the input state and output. Before allowing a retry, the orchestrator compares the current fingerprint against all previous attempts for that step. If the fingerprint matches a previous attempt—meaning the agent is about to retry an identical situation—the retry is blocked.

This is fundamentally different from a retry counter. A counter says "you have N attempts." A fingerprint says "you may not attempt the same thing twice." Combined with progress conditions, this creates a system where every retry must represent genuine forward movement—new information, a modified approach, or a changed context.

4.3 The Trust Problem

The false positive problem is ultimately a trust problem. When an agent reports success, can you trust that report? Our experimental results suggest that for standard approaches, you cannot. Naive prompting reported 100% success on structured extraction tasks while achieving only 34% actual correctness. The system was confident and wrong two-thirds of the time.

This has profound implications for autonomous deployment. A system with a 66% false positive rate cannot be left unsupervised—it requires human review of every output, which defeats the purpose of automation. The false positive rate is not just a quality metric; it is the boundary between "AI assistant that needs babysitting" and "AI agent you can deploy with confidence."

Manifold's external specification-driven validation eliminates this problem. Because the agent does not judge its own output, the trust question shifts from "do I trust the agent?" to "do I trust

the specification?" Specifications are explicit, inspectable, deterministic, and testable—all properties that conversational prompts lack.

5. Results at a Glance

We validated Manifold through 600 controlled trials across four experimental domains, comparing three approaches: naive prompting (single prompt, no orchestration), smart control (retry logic without loop prevention), and Manifold (specification-driven orchestration with loop detection). Full methodology, statistical analysis, and detailed results are available in the accompanying scientific paper.

5.1 Key Findings

Metric	Naive	Smart Control	Manifold
True success (structured tasks)	34%	36%	94%
False positive rate	66%	62%	0%
Field-level accuracy	57.3%	51.6%	99.1%
Cost vs. naive baseline	1.0×	1.5–3.5×	0.36–1.0×
Loop prevention	None	Counter only	Fingerprint + progress

Table 2: Cross-experiment performance summary. Structured task results from EXP3 (email extraction, n=150).

5.2 The False Positive Discovery

The most striking finding emerged from the structured data extraction experiment (EXP3). Under lenient self-validation—where each approach judged its own output—naive prompting reported 100% success. Under strict universal validation—where an external specification evaluated every output against the same criteria—true success was only 34%.

This 66-percentage-point gap is the false positive rate: the proportion of outputs the system believes are correct but actually are not. Smart control's retry logic barely improved matters (62% false positive rate), because retrying the same approach without external verification simply produces more confidently wrong outputs.

Manifold achieved 94% true success with zero false positives. Every output it reported as successful was actually correct. The difference is entirely attributable to external specification-driven validation—the agent was not permitted to judge its own work.

5.3 Cost Efficiency

A common concern with additional validation is cost overhead. Our results show the opposite: Manifold was consistently the most cost-efficient approach across experiments. Smart control's retry logic inflated costs by 1.5 to 3.5 times compared to the naive baseline, because it retried failures without preventing duplicate work. Manifold's loop prevention avoided these wasted retries, achieving up to a 64% cost reduction compared to retry-based approaches while maintaining superior output quality.

In the most adversarial domain tested (DALL-E 3 sprite generation, where the model actively resists structured constraints), smart control spent 3.5 times more per successful output than the naive baseline. Manifold spent less than the baseline while achieving higher success rates, because it never wasted compute on duplicate attempts.

6. What This Means

6.1 The Shift to Trustworthy Automation

The practical implication of zero false positives is not just better accuracy—it is a change in what you can do with AI agents. A system with a 66% false positive rate requires human review of every output. A system with 0% false positives can operate autonomously. This is the difference between AI as a tool that assists humans and AI as an agent that acts independently.

For teams building production LLM systems, the question is not whether to add validation—it is whether your current validation is measuring anything real. If your system reports high success rates without external verification, you likely have a false positive problem you haven't discovered yet.

6.2 Specifications as a Design Practice

Beyond the experimental results, the deeper contribution of this work is a shift in how we think about LLM agent design. The current paradigm treats prompts as natural language instructions that require interpretation. The specification paradigm treats prompts as contracts that require satisfaction.

This shift has practical consequences for daily work. Writing specifications forces you to articulate what "correct" means before you run the agent, not after. It separates the definition of success from the mechanism of execution. It makes failures explicit, diagnosable, and recoverable. And it creates a shared vocabulary between the humans who design workflows and the agents who execute them—not the ambiguous vocabulary of natural language, but the precise vocabulary of contracts.

6.3 When Not to Use Specifications

Specification-driven orchestration is most valuable for structured tasks where correctness can be defined formally: data extraction, format compliance, multi-step workflows with verifiable intermediate outputs. It is less applicable to purely creative tasks where "correct" is subjective, or to exploratory tasks where the output space is not well-defined in advance.

However, even in creative domains, the loop prevention and cost control mechanisms provide value. Our image generation experiments (EXP1 and EXP2) showed that Manifold maintained comparable or superior success rates while preventing the cost inflation that plagues retry-based approaches in adversarial domains.

7. Getting Started

Manifold is open source and available for immediate use. The framework requires Python 3.10+ and has minimal dependencies.

Resources

- **GitHub Repository:** github.com/fabs133/manifold — Source code, examples, and documentation.
- **Scientific Paper:** Full experimental methodology, statistical analysis, and detailed results for all 600 trials across four domains.
- **Architecture Specification:** Detailed technical reference for the manifest schema, specification system, router, and orchestrator.

Core Concepts in Three Minutes

If you take one thing from this whitepaper, let it be this: stop writing prompts as instructions and start writing them as specifications. Define what valid output looks like before you ask for it. Verify output externally, not through self-assessment. Require progress before allowing retries. And make your workflows declarative—the structure should live in data, not in code.

The agents become replaceable components. The specifications remain the laws of physics. And the system becomes something you can trust.

Contact

Fabio-Eric Rempel • fabiorempel@proton.me • github.com/fabs133

© 2026. This work is licensed under CC BY 4.0.